2026年4月26日

OshiruZine-AI

人工知能・機械学習・LLMを中心に、
技術トレンドから実用事例まで幅広く扱う技術系雑誌。

OshiruZine-AI

2026年4月26日 号

目次

1. Claude Code/Codexでのソフトウェア開発の最新ベストプラクティス　*スタンダード（約2000字）*
2. GPT-5・Gemini 2.5・Claude 4——2026年フロンティアモデル徹底比較と選定ガイド　*特集（約4000字）*
3. EU AI Act全面施行から半年——日本企業が今すぐ対応すべきAIガバナンスの実務　*スタンダード（約2000字）*
4. AIエージェント時代の『ツール呼び出し』設計パターン5選　*ショート（500〜1000字）*

Claude Code/Codexでのソフトウェア開発の最新ベストプラクティス

「AIにコードを書かせる」時代は終わった。2026年の今、私たちは「AIと共にソフトウェアを設計し、構築し、運用する」時代の真っただ中にいる。その最前線に立つのが、Anthropicの**Claude Code**とOpenAIの**Codex**だ。両ツールはこの1年で劇的な進化を遂げ、もはや単なるコード補完ツールではなく、アーキテクチャ設計からCI/CDパイプラインの最適化まで担う「開発パートナー」へと変貌した。本コラムでは、2026年4月時点での最新動向を踏まえ、これらのAIコーディングエージェントを実務で最大限に活用するためのベストプラクティスを整理する。

エージェント型開発の現在地

2025年後半から2026年にかけて、AIコーディングツールの勢力図は大きく動いた。Claude Codeは2025年のCLIベースのエージェントとして登場して以降、ターミナル上でファイルの読み書き、テスト実行、Git操作までを自律的にこなす能力を着実に拡張してきた。一方、OpenAIのCodexも2025年5月のリリース以降、クラウドサンドボックス上での並列タスク実行や、リポジトリ全体を文脈として把握するコンテキスト理解の深化で存在感を増している。

2026年に入り、両者に共通する重要なトレンドがある。それは「**指示の精度がアウトプットの品質を決定する**」というシンプルだが本質的な原則がより鮮明になったことだ。モデル性能の向上により「何でもそれなりにできる」状態になったからこそ、人間側の設計力・指示力が差別化要因になっている。

ベストプラクティス①：CLAUDE.mdとシステムプロンプトの戦略的設計

Claude Codeを使う上で最も見過ごされがちだが最も効果の高い施策が、リポジトリルートに配置する**CLAUDE.md**ファイルの最適化だ。このファイルはClaude Codeがタスク実行時に最初に参照するコンテキストであり、いわば「AIチームメンバーへのオンボーディングドキュメント」として機能する。

2026年現在のベストプラクティスとしては、以下を含めることが推奨される。

- **プロジェクトの技術スタック**と各ライブラリのバージョン制約
- **コーディング規約**（命名規則、エラーハンドリングパターン、ログ形式）
- **禁止事項**（使ってはいけないライブラリ、避けるべきパターン）
- **テスト戦略**（ユニットテストの粒度、E2Eテストの実行方法）
- **アーキテクチャ上の制約と意思決定の経緯**（ADR的な記述）

Codex側でも同様に、リポジトリの`AGENTS.md`やシステムプロンプトでプロジェクトのルールを事前定義するアプローチが定着している。鍵は「曖昧さを排除すること」だ。AIは曖昧な指示に対して「最もありそうな」回答を返すが、それがプロジェクト固有の正解とは限らない。

ベストプラクティス②：タスク分割とコンテキスト管理

大規模な機能実装をAIに一括で投げる開発者は未だに多いが、これは2026年でも依然としてアンチパターンだ。Claude Codeの拡張されたコンテキストウィンドウ（現在は最大200Kトークンに対応）やCodexのリポジトリ全体スキャン機能があっても、**タスクの粒度を適切に分割する**ことが品質の鍵を握る。

実践的なアプローチとしては、次のような分割戦略が有効だ。

1. **設計フェーズ**：まずAIにアーキテクチャ案やインターフェース定義を提案させ、人間がレビューする

2. **実装フェーズ**：承認された設計に基づき、モジュール単位でコード生成を依頼する
3. **検証フェーズ**：生成されたコードに対してAI自身にテストを書かせ、実行結果を確認させる

Claude Codeでは`/clear`コマンドによるコンテキストリセットを活用し、フェーズ間で不要な文脈を切り捨てることも重要だ。コンテキストが肥大化すると、初期の指示が「薄まる」現象が発生し、後半の出力品質が低下する。

ベストプラクティス③：ヒューマンインザループの最適配置

2026年のAIコーディングツールは自律性が飛躍的に向上したが、それでも**人間の介入ポイントを意図的に設計する**ことが不可欠だ。Claude Codeの権限設定では、ファイルの書き込みやコマンド実行に対して承認フローを挟むことができる。Codexも同様に、PRの自動生成後に人間レビューを必須とするワークフローが標準化されている。

特に注意すべきは以下の場面だ。

- **セキュリティに関わる変更**（認証・認可ロジック、暗号化処理）
- **データベーススキーマの変更**（マイグレーションの不可逆性）
- **外部APIとの契約に関わるインターフェース変更**
- **パフォーマンスクリティカルなパス**の最適化

「AIが書いたコードだから大丈夫だろう」という信頼は危険だ。AIはテストを通すコードは書けるが、テストが網羅していない暗黙の要件を見落とすことがある。

ベストプラクティス④：CI/CDとの統合による品質ゲート

最新のベストプラクティスとして急速に普及しているのが、AIが生成したコードに対する**自動品質ゲート**の設置だ。具体的には、CI/CDパイプラインにAI生成コ

ード専用のチェックを組み込む手法である。

- 静的解析ツール（ESLint、Ruff、SonarQube等）による自動スキャン
- AI生成コードであることを示すメタデータの付与とトレーサビリティ確保
- セキュリティスキャナによる脆弱性チェックの義務化
- カバレッジ閾値の厳格な適用

Claude CodeもCodexも、テスト実行結果を受けてコードを自動修正するフィードバックループを構築できる。この「生成→テスト→修正」のサイクルを自動化することで、開発速度と品質の両立が現実的になっている。

未来を見据えて

Claude CodeとCodexの競争と進化は、ソフトウェア開発の生産性を確実に押し上げている。しかし、ツールの能力が上がるほど、それを使いこなす人間側の**設計力・判断力・品質への責任意識**がより重要になるという逆説がある。AIは最高の道具だが、良いソフトウェアを作る責任は依然として私たちにある。2026年のエンジニアに求められるのは、「コードを書く力」ではなく「AIと共に正しいソフトウェアを作る力」なのだ。

GPT-5・Gemini 2.5・Claude 4——2026年フロンティアモデル徹底比較と選定ガイド

2026年の春、私たちはついに「どのLLMを選んでも大差ない」という時代の終わりを目撃している。OpenAIのGPT-5、GoogleのGemini 2.5、AnthropicのClaude 4——この三つのフロンティアモデルが出揃った今、それぞれの設計哲学

の違いはかつてないほど鮮明になった。汎用性で圧倒するか、マルチモーダル統合で攻めるか、安全性と精密さで勝負するか。モデル競争が新たなフェーズに突入した今こそ、エンジニアと研究者が「何を基準に選ぶべきか」を冷静に整理するタイミングだ。本稿では、2026年前半時点のベンチマーク、コスト構造、ライセンス形態、そしてユースケース別の選定指針を徹底的に比較する。

2026年前半、フロンティアモデルの勢力図

2025年末から2026年初頭にかけて、主要3社は相次いでフラッグシップモデルを刷新した。OpenAIはGPT-5を2025年後半に段階的にリリースし、推論能力の大幅強化とネイティブなツール利用能力の向上を打ち出した。GoogleはGemini 2.5 Proを2025年3月に公開し、100万トークンを超えるコンテキストウィンドウと「思考モード」による高度な推論を前面に押し出した。AnthropicはClaude 4を投入し、エージェント的な長時間タスク遂行能力と、さらに洗練された安全性フレームワークを武器にエンタープライズ市場を狙っている。

興味深いのは、三者三様の戦略がここにきて明確に分岐している点だ。GPT-5は「プラットフォームとしての総合力」、Gemini 2.5は「Google生態系との統合とマルチモーダル」、Claude 4は「信頼性・安全性・長文理解」——この棲み分けが、技術選定においてむしろ判断をしやすくしている。

ベンチマーク比較：数字が語る実力差

2026年前半の各種ベンチマークを横断的に見ると、以下のような傾向が浮かび上がる。

コーディング能力（SWE-bench、HumanEval系） GPT-5とClaude 4がほぼ拮抗し、業界最高水準のスコアを叩き出している。GPT-5はとりわけ複雑なリファクタリングタスクや大規模コードベースの理解に強く、Claude 4はバグの検出と修正、テストコード生成でやや優位に立つ。Gemini 2.5 Proも健闘しているが、コーディング特化のタスクでは上位2モデルにわずかに及ばない場面が散見される。

推論・数学（GPQA、MATH、ARC-AGI系） ここではGemini 2.5 Proの「思考モード（thinking mode）」が際立つ。段階的な推論プロセスを明示的に展開できるこのモードは、複雑な数学的推論や科学的問題解決において非常に高い精度を見せる。GPT-5のo3相当の推論機能も強力で、特に多段階の論理推論ではトップクラス。Claude 4は純粋な数学ベンチマークでは3番手だが、推論過程の説明可能性（explainability）という観点では他を凌駕する。

長文処理・文書理解 Gemini 2.5が100万トークン超のコンテキストウィンドウを誇り、この分野では頭一つ抜けている。数百ページのPDFを一括で処理し、横断的な質問応答を行うタスクでは、Gemini 2.5の右に出るモデルはない。Claude 4も20万トークンクラスのコンテキストを安定的に扱えるが、Geminiほどの超長文には対応しきれない。GPT-5は12.8万トークン前後で、コンテキスト長では最も控えめだが、与えられた範囲内での情報検索精度は極めて高い。

マルチモーダル（画像・動画・音声） Gemini 2.5がネイティブマルチモーダルの強みを最大限に活かし、この領域では圧倒的だ。画像内のテキスト認識、動画の要約、音声からのリアルタイム書き起こしまで、一つのモデルでシームレスに処理できる点は他社にない強みである。GPT-5もVision機能が強化されているが、動画や音声の統合処理ではGeminiに一歩譲る。Claude 4は画像理解に対応しているものの、動画・音声のネイティブ処理は限定的である。

安全性・ハルシネーション抑制 Claude 4がこの領域でリーダーシップを取っている。Anthropicの「Constitutional AI」の進化版フレームワークにより、指示追従性と安全性のバランスが巧みに設計されている。とりわけ「知らないことを知らないと言える」能力——つまりハルシネーションの自己検知と回避——においてClaude 4は業界最高水準だ。医療・法務・金融といったミッションクリティカルな領域では、この特性が決定的な差別化要因になる。

コスト構造：API価格と実運用コストの現実

2026年4月時点のAPI価格を比較すると、三者のコスト構造にも明確な違いがある。

GPT-5は入力・出力ともにトークン単価がやや高めだが、バッチ処理APIや「GPT-5 mini」系の軽量バリエーションを組み合わせることで実運用コストを抑える戦略が取れる。OpenAIのティアシステムは成熟しており、利用量に応じたボリュームディスカウントも充実している。

Gemini 2.5 Proは、Google Cloudとの統合によるクレジットやディスカウントが大きなアドバンテージだ。特にVertex AI経由での利用は、既存のGCP環境を持つ企業にとってはTCO（総保有コスト）面で有利に働く。長文処理においてはコンテキストウィンドウが大きい分、入力トークン量が増えてコストが膨らむリスクがある一方、コンテキストキャッシュ機能によるコスト削減も可能だ。

Claude 4はAnthropicのAPI経由のほか、Amazon Bedrock、Google Cloud Vertex AIでも利用可能で、マルチクラウド戦略との親和性が高い。トークン単価はGPT-5と同等かやや安い水準で推移しているが、長時間のエージェントタスクでは累積コストに注意が必要だ。

実運用においてはAPI単価だけでなく、「同じタスクを達成するのに何トークン必要か」という効率性が重要になる。Claude 4は指示追従性の高さから、プロンプトの試行錯誤回数が少なく済む傾向があり、実質コストが下がるケースが報告されている。Gemini 2.5は一度に大量の文書を投入して処理できるため、分割処理のオーバーヘッドを削減できる。GPT-5はツール呼び出しの精度が高く、エージェント型ワークフローにおけるリトライコストが低い。

ライセンス・データガバナンスの比較

エンタープライズ導入において避けて通れないのがライセンスとデータガバナンスの問題だ。

OpenAIはビジネスプランおよびエンタープライズプランにおいて、入力データをモデルの学習に使用しないことを明示している。SOC 2 Type II準拠やデータ処理契約（DPA）の提供も標準化されており、コンプライアンス要件の厳しい企業でも導入障壁が低い。

Googleは、Gemini APIのデータ利用ポリシーを無料版と有料版で明確に分けている。有料API（Vertex AI経由）ではデータがモデル改善に使われない保証があり、EU AI Actへの対応も進んでいる。Googleの既存のデータセンターインフラとセキュリティ認証の蓄積は、規制産業への訴求力が高い。

Anthropicは設立当初からの安全性重視の姿勢をライセンス面にも反映させている。利用規約（AUP）は業界でも最も詳細な部類であり、モデルの誤用に対するガードレールが厚い。一方で、この厳格さが一部のユースケース——たとえば創作・エンタメ系やレッドチーム的な利用——では制約と感じられる場面もある。

ユースケース別・選定ガイド

ここからは具体的なユースケースごとに、どのモデルが最適解になりやすいかを整理する。

ソフトウェア開発・コード生成

大規模コードベースを扱う開発支援にはGPT-5が最も安定した選択肢だ。IDE統合（Copilot等）のエコシステムが最も成熟しており、ツール呼び出しとの連携精度も高い。一方、コードレビューやバグ検出に特化するならClaude 4の精密さが光る。個人的な体感として、「コードを書かせる」ならGPT-5、「コードを読ませる」ならClaude 4という使い分けが有効だ。

データ分析・ビジネスインテリジェンス

Google Workspace やBigQueryとの統合を前提とするなら、Gemini 2.5一択と言っていい。大量のスプレッドシートやデータベースクエリ結果を一括で投入し、自然言語で分析指示を出す——このワークフローにおけるGeminiの生産性は圧倒的だ。クラウドに依存しない環境であれば、GPT-5のCode Interpreter相当機能も強力な選択肢となる。

文書処理・法務・コンプライアンス

長文の契約書レビュー、規制文書の解析、社内ポリシーとの整合性チェック——こうしたミッションクリティカルな文書処理では、Claude 4のハルシネーション耐性と引用精度が最大の武器になる。超長文を一括処理する必要がある場合はGemini 2.5の長文コンテキストが有利だが、正確性を最優先するならClaude 4を推奨する。

研究・学術利用

論文の要約、文献レビュー、実験データの解釈といった学術用途では、三者いずれも十分な性能を持つ。ただし、推論過程の透明性と引用の正確さを重視するならClaude 4、大量の論文を横断的に処理するならGemini 2.5、最新の研究ツールやプラグインとの連携を重視するならGPT-5、という優先順位になる。

エージェント型アプリケーション

2026年の最もホットな応用分野であるAIエージェント開発において、三者は異なるアプローチを取っている。GPT-5は関数呼び出し（Function Calling）の精度とOpenAI Agents SDKの充実度で先行している。Claude 4は「コンピュータ使用（Computer Use）」機能の進化により、GUI操作を含む長時間自律タスクに強みを持つ。Gemini 2.5はGoogleのエコシステム（検索、マップ、カレンダー等）とのネイティブ連携で独自の地位を築いている。どのエージェントフレームワークを使うかによっても最適解は変わるが、汎用的なエージェント開発ならGPT-5、特定業務の自動化ならClaude 4、Google環境との統合ならGemini 2.5が第一候補となる。

マルチモーダル・クリエイティブ用途

画像・動画・音声を統合的に扱うアプリケーションでは、Gemini 2.5の優位性が際立つ。リアルタイムの映像解析、音声対話、画像生成指示の組み合わせなど、モダリティを横断するタスクでは現時点でGeminiに代わるモデルはない。

実践的な選定フレームワーク

最後に、技術選定を行う際の判断軸を体系化しておこう。

第1軸：タスク特性——何をさせるのか。コード生成、文書処理、マルチモーダル処理、エージェント型タスクのどれが主か。

第2軸：品質要件——ハルシネーション許容度はどの程度か。ミッションクリティカルな用途ならClaude 4の安全性が効いてくる。

第3軸：スケール要件——処理する文書量やコンテキスト長はどの程度か。超長文処理ならGemini 2.5が不可欠。

第4軸：インフラ制約——既存のクラウド環境はどこか。GCP環境ならGemini、Azure環境ならGPT-5が自然な選択になる。Claude 4はマルチクラウド対応で柔軟。

第5軸：コスト感度——予算制約の厳しさ。軽量タスクには各社のmini/flash系モデルを組み合わせるハイブリッド戦略が有効だ。

そして忘れてはならないのは、**単一モデルに固定する必要はない**という点だ。2026年の成熟したLLMエコシステムにおいては、タスクごとに最適なモデルを使い分ける「マルチモデル戦略」がベストプラクティスになりつつある。抽象化レイヤーを設け、バックエンドのモデルを切り替え可能にしておくアーキテクチャが、長期的な柔軟性を担保する。

おわりに

GPT-5、Gemini 2.5、Claude 4——三つのフロンティアモデルが出揃った2026年前半は、LLM技術選定において最もエキサイティングであると同時に、最も慎重さが求められるタイミングだ。ベンチマークの数字だけでなく、自社のユースケース、インフラ、ガバナンス要件を冷静に見極め、合理的な判断を下してほし

い。モデル競争が激化する今だからこそ、「選ぶ力」こそがエンジニアの最大の武器になる。

EU AI Act全面施行から半年——日本企業が今すぐ対応すべきAIガバナンスの実務

「うちはEU域内に拠点がないから関係ない」——そう思っていた日本企業の開発チームに、2026年初頭、欧州委員会からの質問状が届いた。この話はもはや噂ではない。EU AI Actの域外適用が現実のものとなったいま、日本のAIエンジニア・研究者にとって「対岸の火事」ではいられない状況が到来している。

2025年8月の本格適用から何が変わったか

EU AI Act（人工知能規則）は2024年8月に発効し、段階的に適用範囲が拡大されてきた。2025年2月には禁止AIの規定が施行され、同年8月にはいよいよ高リスクAIシステムに関する義務が本格適用された。そして2026年4月現在、約半年が経過したことで、欧州AI局（AI Office）および各国の市場監視当局が実際の執行に動き始めている。

注目すべきは、域外適用の事例が複数報告されている点だ。EU域内のユーザーに対してAIシステムの出力を提供している場合、開発拠点がEU域外であっても規則の対象となる。2026年第1四半期には、アジア圏のSaaS事業者に対して是正勧告が出されたとの報道もあり、日本企業も無縁ではいられない。

リスク分類——自社のAIシステムはどこに位置するか

AI Actの根幹をなすのがリスクベースのアプローチだ。すべてのAIシステムは以下の4段階に分類される。

1. **禁止リスク（Unacceptable Risk）**：ソーシャルスコアリング、リアルタイム遠隔生体認証（一部例外あり）など。これらは原則としてEU域内での提供が禁止されている。
2. **高リスク（High Risk）**：採用・人事評価、信用スコアリング、教育における成績評価、重要インフラの安全コンポーネントなど、Annex IIIに列挙されたカテゴリ。最も厳格な義務が課される。
3. **限定リスク（Limited Risk）**：チャットボットやディープフェイク生成など。透明性義務（AI生成であることの明示等）が中心。
4. **最小リスク（Minimal Risk）**：スパムフィルターなど。特段の義務はない。

日本の開発チームがまず着手すべきは、自社が提供・開発しているAIシステムをこの分類に当てはめる作業だ。特に、BtoB SaaSやAPI提供を通じてEU域内の顧客にサービスを提供している場合、意図せず高リスクカテゴリに該当するケースが散見される。たとえば、HR Techの採用支援ツールや金融向けの与信判定モデルは典型的な高リスクAIだ。

文書化義務——何をどこまで残すべきか

高リスクAIシステムの提供者（プロバイダー）に対しては、広範な文書化義務が課されている。具体的には以下の項目が求められる。

- **技術文書（Technical Documentation）**：システムの設計仕様、学習データの概要、アルゴリズムの説明、性能評価指標とその結果
- **リスクマネジメントシステム**：開発から運用までのライフサイクル全体を通じたリスク特定・低減措置の記録

- **データガバナンス**：学習データの出所、前処理手順、バイアス検証の方法と結果
- **ログ記録（Logging）**：システムの動作ログを自動的に記録する仕組みの実装
- **適合性評価（Conformity Assessment）**：市場投入前に、自己評価または第三者機関による評価を完了していること

ここで実務上重要なのは、「後から文書を整備する」のでは間に合わないという点だ。開発プロセスの初期段階からドキュメンテーションをワークフローに組み込む、いわゆる"Compliance by Design"の思想が不可欠である。MLOpsパイプラインにモデルカードの自動生成やデータリネージの追跡機能を統合しておくことが、結果的に最もコスト効率が良い。

監査対応——当局からの問い合わせに備える

2026年に入り、欧州AI局は「AI Pact」の自主的なコミットメントから一歩進め、正式な市場監視活動を本格化させている。監査対応で押さえるべきポイントは次の3つだ。

第一に、EU域内の代理人（Authorized Representative）の設置。 域外事業者がEU市場にAIシステムを提供する場合、書面による委任を受けたEU域内の代理人を指定する義務がある。GDPRにおける代理人制度と類似の仕組みだが、AI Actでは対象範囲が異なるため、別途の対応が必要となる。

第二に、インシデント報告体制の構築。 高リスクAIシステムに重大なインシデント（基本的権利の侵害など）が発生した場合、当局への報告義務がある。発生から所定の期間内に報告するためには、社内のエスカレーションフローを事前に設計しておく必要がある。

第三に、適合性評価の証跡管理。 当局は技術文書や適合宣言書の提出を求めることができる。これらがすぐに提出できる状態になっているかどうかが、執行リスクを大きく左右する。

日本国内の動きとの連動

日本政府も手をこまねいているわけではない。2025年に改訂された「AI事業者ガイドライン」は、リスクベースアプローチの考え方をEU AI Actと一定程度整合させている。さらに、2026年度に向けた法制化の議論も進んでおり、広島AIプロセスを起点とした国際的な相互運用性の確保が政策課題となっている。

実務的な観点からは、EU AI Actへの対応をグローバル標準のベースラインとして整備し、日本国内のガイドラインを追加レイヤーとして重ねるアプローチが合理的だ。ISO/IEC 42001（AIマネジメントシステム）の認証取得も、対外的な信頼性を示す有効な手段として注目されている。

まずは「現状把握」から始めよ

AI Actへの対応は一夜にして完了するものではない。だが、最初の一歩は明確だ。自社のAIシステムの棚卸しを行い、リスク分類を確定し、文書化のギャップを特定すること。これだけでも、執行リスクは大幅に低減できる。

規制はイノベーションの敵ではなく、信頼されるAI開発のための共通言語だ。半年後に「あのとき動いておけば」と後悔しないために、今日のスプリントにガバナンスのタスクを1つ追加するところから始めてほしい。

AIエージェント時代の『ツール呼び出し』設計パターン5選

もはやLLMは「賢いテキスト生成器」ではない。2026年現在、MCP（Model Context Protocol）の事実上の標準化とマルチエージェントフレームワークの成

熟により、LLMが外部ツールを自律的に選択・呼び出す「エージェンティックAI」は本番環境の当たり前になった。だが、いざプロダクトに組み込むと設計判断に迷う場面は多い。本コラムでは、現場で頻出する5つのツール呼び出しパターンを紹介する。

パターン1：シングルショット・ディスパッチ

最もシンプルな形。ユーザーの意図を1回のLLM推論で判定し、対応するツールを1つだけ呼び出す。FAQ検索や単機能APIラッパーなど、タスクが明確な場面で有効だ。レイテンシが低く、デバッグも容易なため、まず最初に検討すべきベースラインとなる。

パターン2：シーケンシャル・チェイン

ツールの実行結果を次のLLM推論に渡し、必要に応じて別のツールを連鎖的に呼ぶパターン。「データ取得→加工→可視化」のように依存関係が直列なワークフローに適する。MCP対応クライアントならツール定義の動的差し替えも容易で、ステップごとに利用可能ツールを絞ることで幻覚呼び出しを抑制できる。

パターン3：パラレル・ファンアウト

独立した複数ツールを同時に呼び出し、結果を集約する。例えば複数SaaSのデータを並列取得して統合レポートを生成するケースだ。OpenAIやAnthropicの最新APIが対応する「parallel tool calls」機能を活用すれば、実装コストを大幅に下げられる。タイムアウトとフォールバックの設計が肝になる。

パターン4：プランナー＋エグゼキューター分離

計画フェーズで呼び出すツールと順序をJSON等の構造化プランとして生成し、実行フェーズで忠実に遂行する二層構造。LangGraphやCrewAIの最新版で標準サポートされ、複雑なマルチステップタスクの再現性と監査性が飛躍的に向上す

る。プランの検証レイヤーを挟むことで、意図しないツール連鎖を防止できる点が本番運用で重宝する。

パターン5：ネゴシエーション型マルチエージェント

複数のエージェントがそれぞれ固有のツールセットを持ち、タスクの委譲・交渉を通じて協調するパターン。AutoGen v0.4系やGoogle Agent Development Kitの普及により、エージェント間のプロトコル設計が標準化されつつある。組織横断の業務自動化など大規模シナリオに向くが、オーケストレーションの複雑性は最も高い。

明日から使える実装のヒント

どのパターンを選ぶかは「タスクの複雑度 × 許容レイテンシ × 監査要件」の3軸で判断するとよい。まずはパターン1・2で小さく始め、要件に応じて3〜5へ段階的に拡張するのが失敗の少ないアプローチだ。MCP対応のツール定義を最初から整備しておけば、パターン間の移行コストも最小限に抑えられる。エージェント時代の設計力は、LLMそのものの性能と同じくらいプロダクトの競争力を左右する——今こそ手を動かして試してみてほしい。